

Top 30 Scenario-Based Interview Questions — DevSync AI

"What happens if..." / "What would you do if..."

Questions based on deep analysis of actual project code. Important terms are in bold dark text.

Q1. On every login, your backend calls `prisma.user.findUnique` then `bcrypt.compare`. If your database is slow, every login becomes slow. How would you handle this?

Hinglish: Ye valid concern hai. Abhi mere **login function** mein har baar **`prisma.user.findUnique`** call hoti hai, phir **`bcrypt.compare`** hota hai jo deliberately slow hai **10 salt rounds** ki wajah se. Agar DB slow ho, toh dono milake latency kaafi badh sakti hai. Is problem ko solve karne ke liye main **connection pooling** improve karta — Prisma by default pool maintain karta hai, par uske size ko tune kar sakte hain. Aur bcrypt ke liye, salt rounds ko **10** hi rakhna chahiye — isse kum karna **security risk** hai. Long term mein main **Redis caching layer** add karta jisme verified user ka **userId** thodi der ke liye cache ho sake taaki repeated DB hits avoid hon.

English: This is a valid concern. In my **login function**, every call hits **`prisma.user.findUnique`** followed by **`bcrypt.compare`**, which is intentionally slow due to **10 salt rounds**. If the database is sluggish, combined latency becomes significant. I would tune **Prisma's connection pool size** via the `DATABASE_URL` parameter to allow more concurrent DB connections. The bcrypt cost factor should remain at **10** — lowering it is a **security tradeoff**. Long-term, I would introduce a **Redis caching layer** to cache verified user sessions and reduce repeated database hits.

Q2. Your AI system prompt injects the entire active file's code into every request. What if someone writes a 10,000-line file? The prompt would be massive.

Hinglish: Ye ek real **token limit problem** hai. Mere **chat function** mein system prompt ke andar **`codeBuffer`** directly inject hota hai — agar file bahut badi ho toh **Gemini ka context window** overflow ho sakta hai ya cost aur latency badh sakti hai. Iska solution ye hoga ki main **code truncation** implement karun — pehle **`codeBuffer`** ko check karun, agar characters **5000** se zyada hoon toh sirf **relevant section** bhejun jaise selected lines ya active function. Ya phir **Streaming API** use karun taaki partial response bhi kaam aaye.

English: This is a real **token limit problem**. In my **chat function**, the **`codeBuffer`** is directly embedded into the system prompt. If a file is extremely large, it can overflow **Gemini's context window** or significantly increase latency and API costs. The fix would be implementing **code truncation** — checking **`codeBuffer`** length before injecting, and if it exceeds **5000 characters**, only sending the **relevant section** such as highlighted selection or active function scope. Alternatively, using the **Streaming API** would allow partial responses to remain functional even with large payloads.

Q3. Your `otpStore` is a JavaScript Map stored in RAM. What happens if the server restarts while a user is mid-registration waiting for OTP?

Hinglish: Ye **memoryStore** ka sabse bada weakness hai. Agar server restart ho jata hai toh **otpStore** complete wipe ho jaata hai kyunki ye sirf **RAM mein** hai — koi **disk persistence** nahi hai. Us user ka OTP lost ho jaata hai aur unhe phir se register karna padega. Iska proper solution hoga **Redis** use karna **TTL (Time-To-Live)** ke saath — Redis persist karta hai aur server restart ke baad bhi data available rahega. Filhaal ke liye maine **5-minute OTP expiry** rakhi hai taaki stale data issue minimize ho, par production mein **Redis mandatory** hoga.

English: This is the biggest weakness of **memoryStore**. If the server restarts, the entire **otpStore** Map is wiped because it lives exclusively in **RAM** — there is no **disk persistence**. Any user mid-registration loses their OTP and must restart. The proper production solution is replacing this with **Redis** using a **TTL (Time-To-Live)** equivalent to the OTP expiry. Redis survives server restarts. Currently, I have mitigated stale data with a **5-minute OTP expiry**, but **Redis would be mandatory** in a true production deployment.

Q4. Your workspace files are stored as JSON.stringify inside the description field of the Project table. What if the JSON becomes corrupted or malformed?

Hinglish: Ye **room.handler** ki problem hai jahan **state.files** ko description field mein **JSON string** ke roop mein save kiya jaata hai. Agar JSON malformed ho jaye, toh **JSON.parse** fail hoga. Maine iske liye already ek **try-catch block** rakha hai — agar parse fail ho toh initialFiles empty ho jaata hai aur ek default **index.js** file create ho jaati hai. To **graceful degradation** already implement hai. Lekin ideal solution ye hota ki files ke liye ek **alag database table** banata jaise ek File model with foreign key instead of cramming JSON into a string field.

English: This is a known limitation where **state.files** is persisted as a **JSON string** in the description field. If JSON becomes malformed, **JSON.parse** throws an error. I have already implemented a **try-catch block** — if parsing fails, initialFiles defaults to an empty array and a fallback **index.js** is created. So **graceful degradation** is already in place. However, the ideal architectural fix would be a **dedicated File database table** with a foreign key relation to Project, eliminating the fragile **JSON-in-string antipattern**.

Q5. Your JWT token has an expiry of 1 day. What happens if a user is actively working when the token expires?

Hinglish: Ye **auth.controller** ki problem hai — **expiresIn: 1d**. Jab token expire hota hai, agli API call par **fetchuser middleware** 401 Unauthorized return karega. **Socket.io connection** bhi agle auth check par fail ho sakta hai. User ka unsaved work **in-memory mein hoga** kyunki auto-save sirf disconnect pe hota hai — toh agar frontend handle nahi karta, **data loss ho sakta hai**. Solution: **Refresh Token mechanism** implement karna — ek short-lived **Access Token** (15 minutes) aur ek long-lived **Refresh Token** (7 days). Har expiry pe background mein silently refresh hota.

English: This concerns the **expiresIn: 1d** setting in **auth.controller**. When the token expires, the next API call hits the **fetchuser middleware** which returns 401 Unauthorized. The **Socket.io connection** may also fail on the next auth check. Since **auto-save only triggers on disconnect**, any in-flight changes in the in-memory store could be lost. The correct solution is a **Refresh Token mechanism** — a short-lived **Access Token** (15 minutes) paired with a long-lived **Refresh Token** (7 days), silently refreshing in the background before expiry.

Q6. Your CORS config has a regex allowing all Vercel subdomains. Couldn't an attacker use malicious.vercel.app to bypass CORS?

Hinglish: Bilkul — ye **cors.js** mein ek **security vulnerability** hai. Ye regex kisi bhi **Vercel subdomain** ko allow karta hai, including attacker ka. Correct fix ye hoga ki sirf **specific origin whitelist** karoon jaise **"https://devsync-ai-kappa.vercel.app"** — regex hatao. **CORS** browser-level protection hai, server-level nahi, isliye ye extra layer hai but still important. Is issue ko maine acknowledge kiya hai — production mein regex remove karna ek **pending improvement** hai.

English: Absolutely — this is a **security vulnerability** in **cors.js**. The regex allows any **Vercel subdomain**, including one created by an attacker. The correct fix is to whitelist only the **specific production origin** and remove the regex entirely. While **CORS** is a browser-level enforcement and not a server-level security boundary, weakening it still exposes the API to unauthorized cross-origin requests. Removing the regex is a **pending production improvement**.

Q7. Your code-change Socket event updates the in-memory store on every keystroke. What if 50 users type simultaneously? Won't this cause a race condition?

Hinglish: Ye valid concern hai. **room.handler** mein **file.content** ki direct assignment hai — JavaScript **single-threaded** hai, isliye true race condition nahi hogi kyunki ek time pe ek hi **event loop tick** run hota hai. Lekin 50 users ke saath, last writer wins — jo bhi **latest keystroke** server pe pahuncha, wahi save hoga. Ye essentially **Last Write Wins (LWW)** strategy hai. Production-grade systems mein is problem ko **Operational Transformation (OT)** ya **CRDTs (Conflict-free Replicated Data Types)** se solve karte hain — jaise **Google Docs** karta hai.

English: This is a valid concern. In **room.handler**, **file.content** assignment is direct. However, since JavaScript is **single-threaded**, true race conditions do not occur at the process level. With 50 simultaneous users, the effective strategy is **Last Write Wins (LWW)** — whoever's keystroke arrives at the server last wins. Some intermediate changes may be overwritten. Production-grade collaborative editors solve this with **Operational Transformation (OT)** or **CRDTs (Conflict-free Replicated Data Types)** — the algorithm powering **Google Docs**. This is a recognized future enhancement.

Q8. Your globalLimiter allows 500 requests per 15 minutes per IP. What if your entire team works from one office with a shared NAT IP?

Hinglish: Bilkul sahi observation hai. **rateLimiters.js** mein **max: 500 per IP** hai. Agar ek office ke 20 log same **NAT IP** se kaam kar rahe hoon, toh unka combined request count bahut jaldi **500 hit** kar sakta hai. Fix: rate limiting ko **IP-based se User ID-based** kar dena — **fetchuser middleware** ke baad JWT se **userId** extract karke per-user limit lagana. Ye **express-rate-limit** mein **keyGenerator function** se configure ho sakta hai.

English: Absolutely correct. In **rateLimiters.js**, **max: 500** is per IP. If 20 office users share one **NAT IP**, their combined requests easily hit **500**, triggering the limiter for everyone. The fix is shifting from **IP-based** to **User ID-based rate limiting** — extracting **userId** from the JWT after **fetchuser middleware** and using it as the rate limit key. This is configurable in **express-rate-limit** via the **keyGenerator function**. It is a more equitable strategy for authenticated applications.

Q9. Your Invitation table has a unique constraint on workspaceId and receiverId. What if an owner wants to re-invite someone who previously rejected the invitation?

Hinglish: Ye **schema.prisma** ki constraint hai — **unique([workspaceId, receiverId])**. Ek baar invitation reject ho jaaye, toh status **REJECTED** ho jata hai, lekin unique constraint ki wajah se naya invite **Prisma unique violation error** throw karega. Is case mein main **invitation.controller** mein **upsert** ya update use karta — pehle check karo ki existing rejected record hai, agar hai toh **status reset** karo **PENDING** pe, naya record create mat karo. Filhaal ye edge case handle nahi hua hai — ye ek **known gap** hai.

English: This is the **unique([workspaceId, receiverId])** constraint in **schema.prisma**. Once an invitation is rejected, the constraint prevents creating a new invitation row for the same pair — Prisma throws a **unique constraint violation**. The correct fix in **invitation.controller** would be an **upsert** — check if a rejected record exists, and if so, reset its status back to **PENDING** instead of creating a new row. This edge case is currently unhandled — it is a **recognized production gap**.

Q10. The description field in Project model is Prisma String mapping to PostgreSQL TEXT. What if someone stores gigabytes of code there?

Hinglish: PostgreSQL mein **String type TEXT column** map hota hai jisme theoretically **unlimited data** store ho sakta hai. Aur **activeWorkspaces Map** mein bhi koi size limit nahi hai. Practically, agar koi bahut bada codebase store kare toh **DB row size** badh jaayegi aur **auto-save query slow** ho jaayegi. Fix: application level pe **file size validation** add karna — agar total serialized files ka size **1MB** se zyada ho toh save refuse karo ya compress karo. Ya proper **object storage** use karo jaise **AWS S3** aur DB mein sirf file path store karo.

English: In PostgreSQL, Prisma String type maps to a **TEXT column** — no practical size limit. The **activeWorkspaces Map** also has no size guard. Storing a very large codebase would bloat the **DB row size** and slow down the **auto-save query**. The fix is adding **application-level file size validation** — if the serialized files string exceeds **1MB**, reject or compress. The architectural improvement would be using **object storage** such as **AWS S3** for file content and storing only a file path reference in the database.

Q11. Your auto-save only triggers when the last user disconnects. What if the user's browser crashes before the disconnect event fires?

Hinglish: Ye ek real **data loss risk** lagta hai. Lekin browser crash ya sudden network cut mein **disconnect event server side pe eventually fire** hota hai — Socket.io ka **heartbeat/ping mechanism** detect kar leta hai ki client gone hai typically **20-30 seconds** mein. Toh theoretically **auto-save trigger** hoga. Lekin agar **server bhi** is window mein crash ho jaaye, toh data lost. Mitigation: **periodic auto-save** implement karna — har **30 seconds** mein background mein RAM se DB sync karna, sirf disconnect pe nahi.

English: This appears to be a data loss risk. However, when a browser crashes, the **disconnect event still fires server-side** because Socket.io's built-in **heartbeat/ping mechanism** detects the dropped connection within approximately **20-30 seconds**. So **auto-save would still trigger**. The real risk is if the **server itself crashes** within that detection window. The mitigation is implementing **periodic background auto-save** — flushing RAM state to the database every **30 seconds** regardless of disconnects, not waiting solely for the last user to leave.

Q12. Your Socket.io auth middleware reads JWT from the handshake. What if someone connects directly via a raw WebSocket tool with a stolen valid JWT?

Hinglish: Ye **sockets/index.js** ka scenario hai. Agar koi raw WebSocket tool se direct connection try kare bina valid JWT ke, toh middleware mein **jwt.verify** fail hoga aur connection reject ho jaayega. Lekin agar koi **valid JWT le le** jaise apna account banake, woh directly Socket events inject kar sakta hai. Isi liye **join-room event** mein maine **database-level authorization check** rakha hai — sirf **workspace members** hi join kar sakte hain. Ye **defense in depth** strategy hai — do layers of verification.

English: In **sockets/index.js**, the auth middleware verifies JWT from the handshake. Without a valid JWT, **jwt.verify** fails and the connection is rejected. However, with a **valid JWT** obtained from a legitimate account, an attacker could inject Socket events directly. This is why I implemented a **database-level authorization check** inside **join-room** — it verifies the authenticated user is actually a **workspace member**. This is the **defense in depth** strategy — authentication at the transport layer, authorization at the business logic layer.

Q13. Your bcrypt uses 10 salt rounds. A teammate changes it to 5 for speed. What happens to existing users trying to login?

Hinglish: bcrypt ek **self-describing format** use karta hai — hashed password ke andar hi **salt rounds** ki information embedded hoti hai. Isliye existing users ke passwords jo **10 rounds** se hashed hain, unpe **bcrypt.compare** correctly kaam karta rahega — chahe code mein rounds 5 kar do. **bcrypt.compare** password ke existing hash ko read karta hai aur **khud hi rounds detect** kar leta hai. Naye users **5 rounds** se hash honge (weaker), purane users **10** se (stronger). Login breaks nahi hoga, lekin ye ek **security regression** hai.

English: bcrypt uses a **self-describing hash format** — the number of **salt rounds** is embedded inside the hash string itself. Therefore, **bcrypt.compare** reads the rounds directly from the stored hash and verifies correctly, regardless of the current code-level setting. Existing users hashed with **10 rounds** continue to log in successfully. Only new users would be hashed with the weaker **5 rounds**. This creates a **security inconsistency** — no login breakage occurs, but the change is still a **security regression** that should be reverted.

Q14. Your Gemini API key is in .env. What if .env accidentally gets pushed to GitHub?

Hinglish: Ye ek critical **secret exposure** risk hai. Agar .env GitHub pe push ho jaaye, toh attacker **Gemini API key** use karke unlimited AI calls kar sakta hai — **billing tumhare account pe** hogi. Iske liye maine **.gitignore** mein .env add kiya hua hai. Prevention ke liye **GitHub Secret Scanning** enable karna chahiye — GitHub automatically detect karta hai agar koi API key commit ho. Aur agar leak ho bhi jaaye, **immediately rotate the key** Google AI Studio se — old key instantly invalidate ho jaati hai.

English: This is a critical **secret exposure** risk. If .env gets pushed to GitHub, an attacker can use the **Gemini API key** to make unlimited AI calls, all billed to your account. I have .env listed in **.gitignore** to prevent this. Additional best practices: enable **GitHub Secret Scanning**, which automatically alerts when API keys appear in commits. Apply the **principle of least privilege** — restrict the key to requests from your server IP. If exposure occurs, **immediately rotate the key** from Google AI Studio — all calls with the old key are instantly invalidated.

Q15. Your WorkspaceMember uses a composite primary key on workspaceId and userId. What if a migration accidentally drops this constraint?

Hinglish: Agar **composite primary key** drop ho jaaye toh ek user ko ek hi workspace mein **multiple times add** kiya ja sakta hai — duplicate rows. Ye member listing mein weird results dega aur **role confusion** create ho sakta hai. Fix: Prisma migrations ko hamesha **create-only flag** ke saath review karna phir apply karna. **prisma migrate diff** se schema changes review karo apply se pehle. Production mein **never run prisma db push** directly — hamesha proper **versioned migrations** use karo.

English: If the **composite primary key** is dropped, a user could be added to the same workspace **multiple times** — creating duplicate rows with potentially conflicting roles. The fix: always generate migrations with the **create-only flag**, review the generated SQL, then apply. Use **prisma migrate diff** to audit schema changes before deploying. In production, **never use prisma db push** — it bypasses migration history. Always use proper **versioned migrations**.

Q16. Gemini has max_tokens set to 2048 for chat. What if the complete answer genuinely requires 5000 tokens?

Hinglish: **ai.controller.js** mein **max_tokens: 2048** hard-coded hai. Agar answer **5000 tokens** ka ho toh response **beech mein cut** ho jaayega — incomplete code ya mid-sentence answer. Solution: pehla — **max_tokens** ko aur badhao, Gemini 3.6 Flash maximum **8192 output tokens** support karta hai. Doosra — **Streaming API** implement karo jisme partial tokens real-time mein user ko dikhao. Teesra — frontend mein **truncation notice** add karo.

English: In **ai.controller.js**, **max_tokens: 2048** is hardcoded. If a complete answer requires **5000 tokens**, the response **truncates mid-way** — incomplete code or a sentence cut off. Solutions: first — increase **max_tokens**, Gemini 3.6 Flash supports up to **8192 output tokens**. Second — implement the **Streaming API** to deliver partial tokens in real-time as they are generated. Third — append a visible **truncation notice** to the frontend message.

Q17. The activeWorkspaces Map grows as users join. Is there any cleanup? What if 10,000 workspaces are simultaneously active?

Hinglish: **memoryStore.js** mein ek **30-minute cleanup interval** hai jo abandoned workspaces delete karta hai — jahan **roomUsers empty** ho. Lekin agar genuinely **10,000 workspace** simultaneously active hoon, toh **Node.js heap memory** bahut zyada consume hogi. Aur main problem: Map ek **process-local structure** hai — **horizontal scaling** multiple servers ke saath different server pe users collaborate nahi kar sakte. Correct solution: **in-memory Map ko shared Redis store se replace** karo taaki true **horizontal scalability** achieve ho.

English: In **memoryStore.js**, a **30-minute cleanup interval** removes abandoned workspaces where **roomUsers is empty**. However, with **10,000 genuinely active workspaces**, **Node.js heap memory** would be severely strained. The deeper issue: Map is **local to one Node.js process** — with **horizontal scaling** across multiple servers, each server has an isolated Map and users on different servers cannot collaborate. The correct solution is **replacing the in-memory Map with a shared Redis store** to enable true **horizontal scalability**.

Q18. Your LeetCode proxy makes direct HTTP requests to LeetCode's internal GraphQL API. What if LeetCode changes their schema or adds bot protection?

Hinglish: Ye ek **third-party dependency risk** hai. **leetcode.controller** mein LeetCode ke internal, undocumented **GraphQL endpoint** pe query jaati hai. Agar LeetCode **schema change** kare ya **bot detection** jaise Cloudflare challenge add kare, problem fetch karna fail ho jaayega. Mitigation: pehla — **try-catch already hai** toh error case mein frontend **graceful message** dikhata hai. Doosra — LeetCode problems ko apne DB mein **cache** karo. Teesra — **official API alternatives** explore karo.

English: This is a **third-party dependency risk**. **leetcode.controller** queries LeetCode's internal, undocumented **GraphQL endpoint**. If LeetCode changes their **schema** or adds **bot detection** such as Cloudflare challenges, problem fetching breaks entirely. Mitigations: first — existing **try-catch** ensures the frontend receives a **graceful error message**. Second — implement **problem caching**, fetch once and store in your database, serving from cache on subsequent requests. Third — migrate to LeetCode's **official API** or use a **licensed third-party problems service**.

Q19. Your WorkspaceRole enum has VIEWER as an option, but does your checkMembership middleware differentiate between COLLABORATOR and VIEWER?

Hinglish: Ye ek **incomplete implementation** hai. **schema.prisma** mein **VIEWER role defined** hai, par **checkMembership.js middleware** sirf check karta hai ki user workspace member hai ya nahi — **role-based permission granularity** implement nahi ki hai VIEWER vs COLLABORATOR ke beech. Matlab agar koi **VIEWER role** pe hai, wo bhi code edit kar sakta hai — jo intended behavior nahi hai. Fix: **checkMembership** mein **requiredRole parameter** add karo aur user ki **WorkspaceRole** ko DB se fetch karke match karo.

English: This is an **incomplete implementation**. The **VIEWER role** is defined in **schema.prisma**, but **checkMembership.js** only validates workspace membership — it does **not enforce permission granularity** between COLLABORATOR and VIEWER. A user assigned **VIEWER role** can still make code edits. This is a **recognized production gap**. The fix: enhance **checkMembership** to accept a **requiredRole parameter**, fetch the user's actual **WorkspaceRole** from the database, and compare it before granting access.

Q20. Your sendOTP calls the Brevo API. What if Brevo is down for maintenance? The user cannot register.

Hinglish: **auth.controller** mein agar **sendOTP fail** hota hai toh OTP delete hota hai aur **400 error** return hota hai — user register nahi kar sakta. Brevo ki **Service Level Agreement (SLA)** 99.9% uptime hai — but zero downtime nahi. Production-level fix: pehla — **fallback email provider**, agar Brevo fail ho toh secondary provider jaise **SendGrid** try karo. Doosra — **queue-based email delivery**, email job ko queue mein dalo aur failed jobs **retry** ho sake. Teesra — **alerting setup** karo. Filhaal ye **single point of failure** hai.

English: In **auth.controller**, if **sendOTP fails**, the OTP is deleted and a **400 error** is returned — the user cannot register. Brevo's **SLA is approximately 99.9% uptime** but not zero downtime. Production-grade fixes: first — **fallback email provider**, if Brevo fails retry with **SendGrid**. Second — **queue-based email delivery**, push jobs to Bull/RabbitMQ and **retry on failure**. Third — set up **failure alerting** for immediate ops notification. Currently this is a **single point of failure** in the registration flow.

Q21. UUID collisions are theoretically possible. How would you handle one?

Hinglish: UUID v4 mein collision probability astronomically low hai — **2 raised to 122** possible values. Practically **near-impossible**. Lekin agar theoretically collision ho, PostgreSQL ka **PRIMARY KEY constraint** automatically **duplicate key violation error** throw karega — Prisma isse **PrismaClientKnownRequestError** ke roop mein expose karega. Main is specific error code ko catch karke **retry-with-new-UUID loop** add kar sakta hoon. Honestly, **UUID collision handling is theoretical**, not practical — isse over-engineer karna nahi chahiye.

English: UUID v4 has a collision probability of approximately **1 in 2 raised to 122** — statistically negligible in any realistic system. Practically, UUID collisions are **near-impossible**. If a theoretical collision occurred, PostgreSQL's **PRIMARY KEY constraint** would throw a **duplicate key violation**, which Prisma surfaces as **PrismaClientKnownRequestError**. I could catch this specific error and implement a **retry-with-new-UUID loop**, but this would genuinely be over-engineering — UUID collision handling is **a theoretical concern, not a practical one**.

Q22. In your execute function, temperature 0.0 is meant for deterministic output. But what if the same code gives different output on different runs?

Hinglish: temperature: 0.0 LLMs mein **greedy decoding** enable karta hai — mathematically most probable next token select hota hai. Par LLMs fundamentally **non-deterministic systems** hain — same prompt pe **different server-side sampling** due to floating point precision differences ya model version updates se slightly different output aa sakta hai. Matlab complex code edge cases mein AI **hallucinate** kar sakta hai ya wrong output de sakta hai. Isliye ye system ek **Virtual Execution Engine** hai, real compiler nahi — critical code ke liye real execution environment use karo.

English: Setting temperature: 0.0 enables **greedy decoding** — the mathematically most probable next token is always selected. However, LLMs are fundamentally **non-deterministic** — the same prompt can yield slightly different outputs across runs due to **floating-point precision differences** in inference hardware or model version updates. For complex edge cases, the AI may **hallucinate incorrect output**. This is precisely why the system is called a **Virtual Execution Engine**, not a real compiler — it is explicitly positioned as a simulation for standard algorithmic problems, not a replacement for actual runtime environments.

Q23. Your globalLimiter uses in-memory storage. What if you run two backend instances behind a load balancer?

Hinglish: Ye rateLimiters.js ki badi problem hai. **express-rate-limit** by default **in-memory MemoryStore** use karta hai — har Node.js instance ki apni alag **rate limit count** hoti hai. Agar load balancer do instances pe requests distribute kare, toh ek user dono instances pe separately **500 requests each** kar sakta hai — effectively **1000 requests total**. **Rate limiting completely broken** ho jaati hai. Fix: **redis-rate-limit store** use karo taaki count ek **shared Redis instance** mein maintain ho.

English: This is a major weakness in rateLimiters.js. **express-rate-limit** uses **in-memory MemoryStore** by default — each Node.js instance maintains its own isolated **rate limit count**. With two instances behind a load balancer, a user gets **500 requests** to each — **1000 total** before being limited. **Rate limiting becomes completely ineffective**. The fix: configure express-rate-limit with a **Redis store** — all server instances share the same counter, making rate limiting work correctly across a **horizontally scaled** deployment.

Q24. The user's JWT is stored in localStorage. What is the XSS risk and why did you accept this tradeoff?

Hinglish: Ye ek classic security debate hai. **localStorage XSS (Cross-Site Scripting) attacks** ke liye vulnerable hai — agar kisi bhi third-party script ko XSS mil jaaye, wo localStorage se **JWT steal** kar sakta hai. Safer alternative hai **httpOnly Cookie** — JavaScript se accessible nahi hai, **XSS proof** hai. Maine localStorage isliye use kiya kyunki ye simpler implementation hai **SPA** ke liye aur httpOnly cookies ke saath **CORS configuration zyada complex** hoti hai. Ye ek **acknowledged tradeoff** hai — production mein **httpOnly Secure SameSite Cookie** mandatory hoga.

English: This is a classic security debate. **localStorage** is vulnerable to **XSS (Cross-Site Scripting)** — any injected malicious script can **steal the JWT** from storage. The safer alternative is an **httpOnly Cookie**, inaccessible to JavaScript entirely, making it **XSS-proof**. I used localStorage for simplicity — it is straightforward for **SPAs** and avoids the complex **credentials: true CORS configuration** required by cookies. This is an **acknowledged tradeoff** — migrating to **httpOnly Secure SameSite=Strict Cookies** would be the mandatory production improvement.

Q25. Your cleanupExpiredOTPs runs via setInterval every 2 minutes. What if heavy server load causes this interval to fire late?

Hinglish: **memoryStore.js** mein **setInterval** hai jo har **2 minutes** mein **cleanupExpiredOTPs** run karta hai. JavaScript **setInterval best-effort** hai — heavy event loop ke case mein delay ho sakta hai. Matlab kuch expired OTPs thodi der **RAM mein reh** sakti hain. **Security risk minimal** hai — kyunki OTP verify hone ke time bhi **Date.now()** aur **storedData.expiresAt** ka comparison hota hai **verifyOtp handler** mein, toh expired OTP accept nahi hoga. Ye cleanup more about **memory hygiene** hai, security nahi.

English: In **memoryStore.js**, **setInterval** runs **cleanupExpiredOTPs** every **2 minutes**. JavaScript **setInterval is best-effort** — under heavy event loop pressure, the callback may fire late, leaving expired OTPs in RAM slightly longer. The **security impact is negligible** — because the **verifyOtp handler** independently compares **Date.now()** against **storedData.expiresAt** before accepting any OTP. The interval is purely for **memory hygiene**, not security. For precision under heavy load, **Redis TTL** would be more accurate.

Q26. Your prisma.js creates a singleton Prisma client. In development, hot-reloading can create multiple instances and exhaust the connection pool.

Hinglish: Ye **lib/prisma.js** ka known **Node.js aur Prisma** issue hai. Development mein **nodemon** har file save pe module re-require karta hai — naya **PrismaClient instance** create hota hai. PostgreSQL connections **pool-based** hain — har instance apna pool maintain karta hai. Bahut saare instances = **connection pool exhaustion** = initialization errors. Fix: **global.__prisma singleton pattern** — pehle check karo **global.__prisma** exists karta hai ya nahi, nahi toh naya banao aur global pe attach karo. **Production** mein ye issue nahi hota.

English: This is a well-known **Node.js and Prisma** issue in **lib/prisma.js**. In development, **nodemon** re-requires modules on every file save — creating new **PrismaClient instances**. Each instance maintains its own PostgreSQL **connection pool**. Many accumulated instances cause **connection pool exhaustion** and initialization errors. The fix is the **global.__prisma singleton pattern** — check if **global.__prisma** already

exists; if yes, reuse it; if no, create a new client and attach it to global. This is a development-only issue — **production processes do not hot-reload.**

Q27. The onDelete Cascade is set on WorkspaceMember and Invitation to User. What happens if a workspace owner's account is deleted?

Hinglish: `schema.prisma` mein **WorkspaceMember** aur **Invitation** dono ke relations mein **onDelete: Cascade** hai User ki taraf se. Agar user account delete ho, toh automatically uske saare **WorkspaceMember records** aur **Invitation records** bhi delete ho jaate hain — **database integrity maintained** rehti hai. Lekin agar deleted user kisi workspace ka **OWNER** tha, toh **Project bhi cascade se delete** ho jaayega — saare collaborators ka access bhi khatam. Ye **data loss risk** hai. Improvement: OWNER delete se pehle **ownership transfer** enforce karo.

English: In `schema.prisma`, both **WorkspaceMember** and **Invitation** relations have **onDelete: Cascade**. If a user account is deleted, all their **WorkspaceMember records** and **Invitation records** auto-delete — **database integrity is maintained**. However, if the deleted user was a workspace **OWNER**, the **Project itself also cascades and deletes** — instantly removing all collaborators' access and their code. This is a **data loss risk**. The improvement: enforce an **ownership transfer** flow before allowing account deletion, or transition orphaned projects to a preserved state rather than hard-deleting.

Q28. Your Gemini API has usage quotas. What happens when you hit the free tier limit mid-conversation?

Hinglish: Jab Gemini free tier quota exhaust ho jata hai, API **429 Too Many Requests** return karta hai. Mere **ai.controller** mein generic catch block hai jo error catch karke **system exception message** return karta hai — user ko specific reason nahi milta. Improvement: **error.status 429** specifically catch karo aur user ko clear message do jaise **"AI usage limit reached, please try again in a minute."** Aur **exponential backoff** implement karo automatic retry ke liye. Long term: **paid tier** mein upgrade karo ya **multiple API keys rotate** karo.

English: When Gemini's free tier quota is exhausted, the API returns **429 Too Many Requests**. My **ai.controller** has a generic catch block returning a system exception message — the user receives no actionable information. Improvement: explicitly check **error.status 429** and surface a clear message such as **"AI usage limit reached, please try again shortly."** Additionally, implement **exponential backoff** for automatic retries. Long-term solutions: upgrade to the **paid tier**, implement **round-robin across multiple API keys**, or add a **request queue with backpressure** management.

Q29. Your join-room Socket handler does a `prisma.project.findUnique` DB call every time someone joins. With 100 users joining simultaneously, is this a bottleneck?

Hinglish: `room.handler` mein **join-room event** pe hamesha **DB call** hoti hai workspace membership verify karne ke liye. 100 simultaneous joins = **100 concurrent DB queries**. Prisma's **connection pool** default approximately **5 connections** se zyada queries aayi toh queue ho jaayenge — **latency spike**. Fix: **membership caching** — recently verified **workspaceId** aur **userId** pairs ko short-lived cache mein rakho jaise **Redis** ya **in-process Map with TTL**. Membership frequently change nahi hoti, isliye ye optimization **high-traffic scenarios** mein significant hoga.

English: In `room.handler`, every **join-room event** triggers a **prisma.project.findUnique DB call** to verify workspace membership. With 100 simultaneous joins, that is **100 concurrent DB queries**. Prisma's default **connection pool** of approximately **5 connections** would queue the excess — causing visible **latency spikes**. Fix: introduce **membership caching** — store recently verified **workspaceId and userId pairs** in a short-lived cache using **Redis or an in-process Map with TTL**. Since membership does not change frequently, a 5-minute cache with **cache invalidation on member removal** would dramatically reduce DB load.

Q30. Your entire backend is a single Node.js process. If an unhandled exception crashes it, all active users lose their sessions. How resilient is your current setup?

Hinglish: `server.js` mein maine **process.on unhandledRejection** aur **process.on uncaughtException** handlers add kiye hain — ye crash ko **log** karte hain. Lekin `uncaughtException` ke baad process ko **exit** karna safer hai kyunki app **inconsistent state** mein ho sakta hai. **Render.com** pe deployment hai — Render automatic **process restart** karta hai crash hone pe. Lekin restart ke time mein jo users active the unka **in-memory code lost** ho jaayega. Resilience improve karne ke liye: **PM2 with cluster mode** use karo aur **30-second periodic auto-save** implement karo.

English: In `server.js`, I have registered **process.on unhandledRejection** and **process.on uncaughtException** handlers that log errors. However, after `uncaughtException`, the process should **exit** — the app may be in an **inconsistent state**. On **Render.com**, the platform **automatically restarts** crashed processes. However, during the restart window, any **in-memory code** for active workspace sessions is permanently lost. Resilience improvements: use **PM2 with cluster mode** to utilize multiple CPU cores so a crash in one worker does not kill all users, and implement **30-second periodic auto-save** so in-memory state is already persisted before any crash window.

All questions grounded in actual code from: `ai.controller.js`, `auth.controller.js`, `memoryStore.js`, `room.handler.js`, `rateLimiters.js`, `schema.prisma`, `cors.js`, `fetchuser.js`, `lib/prisma.js`